

From demonstrations to interactive imitation

Collect data. Learn a policy. Diagnose distribution shift.

In this assignment you will build and compare behavior cloning (BC) and DAgger across control and combinatorial decision making. The environment adapters, interactive interfaces, teachers, network, storage, and experiment runner are supplied. Your implementation work is concentrated in five functions.

Scope and workload

50 demonstrations per Lander variant and 10 Chess games; 110 total. Budget approximately 3-6 hours of active work. Chess games record up to 20 White moves and then a bot takes over. CPU training is sufficient. The time budget is an estimate and depends on game familiarity; no target win rate is required.

Variant	One demonstration	Main learning challenge
Lunar Lander: discrete	An episode; at most 600 physics steps / 300 decisions	Four action classes; compounding control errors
Lunar Lander: continuous	Same episode cap and decision frequency	Two graded controls; regression and actuator dead zones
Chess	20 White moves; automatic Black replies	Sparse coverage of structured positions and move choices

Your deliverables

Code: complete student/algorithms.py. **Evidence:** three frozen demonstration banks, experiment manifests, raw results, and plots. **Report:** 4-6 pages analyzing quantity, action representation, and DAgger. Include a reproduction appendix outside the page limit.

Suggested active-time budget	Target
Setup, reading, and practice	20-40 minutes
Collect three banks (50 + 50 + 10 demonstrations)	20-30 minutes
Implement, test, and run experiments	90-150 minutes
Analyze results and write report	60-120 minutes

Learning target: explain why low supervised loss does not guarantee high rollout return, why feedback on learner-visited states can help, and why neither more data nor DAgger guarantees improvement in every finite experiment.

A demonstration is a trajectory

Practice into a separate folder, then keep all completed collection episodes, including failures. Do not cherry-pick successful landings or winning openings. You play White in Chess. Short early endings still count as one demonstration; report their frequency.

```
ilr collect --domain lander-discrete --data data/lander-discrete
ilr collect --domain lander-continuous --data data/lander-continuous
ilr collect --domain chess --data data/chess
```

Lander commands default to 50 episodes each; Chess defaults to 10 games. Press **Enter** to begin an episode and **Esc** to stop. Repeating the same command resumes by skipping saved seed-numbered files. Unfinished prefixes are discarded. A completed prefix is saved before bot takeover begins.

Variant	Interactive controls
Lander discrete	Up: main engine. Left/right: corresponding jet. No key: coast. Main has priority.
Lander continuous	Hold the mouse in the control pad: vertical main throttle, horizontal side throttle. W/S and A/D ramp controls; Space cuts main.
Chess	Click source and destination. Q/R/B/N chooses promotion. Only legal moves are accepted.

Human labels and assistance

H accepts one teacher suggestion and marks that row assisted. Report the assisted fraction. The headless `--mode teacher` option is for infrastructure smoke tests; these synthetic banks do not satisfy human collection. Do not use `--allow-teacher` for the required experiment.

Prefix versus continuation

Chess records up to 20 **student-side moves**, not 20 total plies. Black uses Stockfish with seeded occasional random opening moves to provide variation. After the prefix, Stockfish takes over. Games stop after at most 160 plies; natural endings can occur earlier.

Do not learn from the takeover suffix

Training uses only the recorded human prefix. The eventual win/draw/loss measures a **hybrid policy: learned or human prefix + bot continuation**. It is useful but does not measure unaided full-game skill.

Freeze the bank before making subsets

Complete and freeze each bank first: 50 files per Lander variant and 10 files for Chess. The loader shuffles complete episodes once with a fixed selection seed. Lander uses nested subsets of 5, 10, and 50 demonstrations; Chess uses nested subsets of 5 and 10 games. Do not split adjacent transitions from the same episode between training and evaluation. Later additions to a bank change its permutation.

What does the learner observe?

Variant	Input x	Output and restrictions
Lander discrete	8 numbers: position, velocity, angle, angular velocity, and two leg-contact flags	4 logits; select one engine action
Lander continuous	The same 8 observation components	2 tanh outputs in $[-1,1]$; main and side controls
Chess	1,032 features from the fully visible position and counters	2,034 logits; mask to legal chess moves

The supplied network is an MLP with two 64-unit ReLU hidden layers. Board features are 16 x 64 planes plus 8 global values. The fixed output layout contains 1,968 geometric moves/promotions and 66 unused slots, which are always masked out. Two input planes and one global input are fixed zeros. Only legal Chess moves can be selected. See `encode_board` and `Policy` for the exact layout.

Chess exposes the current board, but its feature vector omits full repetition history even though the rules engine tracks it. The fixed representation limits the conclusions you can draw.

Inspect one episode and measure its size

```
import numpy as np
with np.load("data/chess/episode_00000.npz", allow_pickle=False) as z:
    print(z["obs"].shape, z["actions"].shape, z["masks"].shape)
    print(z["sources"], z["metadata"])
```

Fields include observations, teacher/demonstrator targets, masks, executed actions, phase, source, and metadata. For 20 board decisions, observations alone use $20 \times 1,032 \times 4 = \mathbf{82,560 \text{ bytes}}$; Boolean masks add **40,680 bytes** before compression. Report actual bank sizes and decision counts. More demonstrations do not imply equal amounts of data across variants.

Chess increases combinatorial and action-selection difficulty. Do not claim its mathematical state-space cardinality exceeds that of a continuous system merely because its input vector is larger.

Fit actions from demonstrations

Let D contain N labeled decisions (x_i, a_i^*, m_i) , where m identifies the action choices available at collection time. BC minimizes a supervised loss over this fixed dataset. For Lander, the provided driver fits normalization statistics using only the selected training subset. Board features already have fixed scales.

1. bc_loss(predictions, targets, masks, continuous) [15]

Continuous: use mean squared error across the batch and both action dimensions. Predictions and targets have shape $[B, 2]$. The returned loss is a scalar tensor supporting backpropagation.

$$L_{\text{cont}} = (1 / 2B) \sum_{i=1}^B \sum_{j=1}^2 (\pi_{\square}(x_i)_j - a_{ij}^*)^2$$

Discrete: predictions are raw logits $[B, A]$, targets are class indices $[B]$, and masks are Boolean $[B, A]$. Set disallowed logits to negative infinity **before** cross-entropy. Do not apply softmax first. Every provided training mask contains its target.

$$L_{\text{disc}} = -(1 / B) \sum_{i=1}^B \log [\exp(z_{i,a^*}) / \sum_{a \in A(m_i)} \exp(z_{i,a})]$$

Explain why masking only the selected action at inference is insufficient. A logit for an unavailable move must not consume probability mass in the training objective.

2. train_bc(policy, data, *, epochs, batch_size, lr, seed) [15]

Train with shuffled minibatch Adam using `bc_loss`. Set training mode, clear gradients, backpropagate, and step the optimizer. Include the final smaller batch. Shuffle reproducibly using a seeded Torch generator. Return one sample-weighted mean loss per epoch. Do not reset weights or normalizer when called again: DAgger warm-starts the same policy.

```
python -m pytest tests/test_algorithms.py -q
ilr train --domain lander-discrete --data data/lander-discrete \
  --demos 5 --out runs/debug-bc
ilr evaluate --checkpoint runs/debug-bc/policy.pt \
  --split validation --episodes 5 --out runs/debug-validation.json
```

Supervised fit and rollout performance differ

A small error changes the next observation, which can lead to further errors outside the demonstration distribution. Conversely, disagreement with a teacher can still produce a reasonable action. Compare losses, teacher disagreement, and actual rollouts; do not treat them as interchangeable. [1,2]

Continuous nuance: main-engine outputs below zero are off; nonnegative outputs correspond to 50-100% thrust. Side thrust has a dead zone. Equal differences in numerical action need not produce equal changes in physical behavior. [3]

Label the states the learner visits

Initialize with BC-50 for Lander or BC-10 for Chess. In each round, roll out a mixture of the current learner and a supplied teacher, obtain the teacher action at every visited input, aggregate the new labels with all previous data, and train again. The supplied runner handles checkpoints, fresh seeds, and evaluation. [2]

3. `beta_schedule(round_index)` [5]

Return the probability of **executing** the teacher action. Indices are zero-based: $\beta_i = 0.5^{i+1}$, giving 0.5, 0.25, and 0.125. Reject a negative index. This changes who acts; the teacher remains the source of every new training target.

4. `collect_dagger_episode(env, policy, beta, rng)` [20]

The environment has already been reset. Repeat the following until `env.prefix_done`; do not reset it inside your function.

Order	Required operation
1	Read obs, mask = <code>env.observe()</code> ; preserve this pre-action input.
2	Query <code>env.expert_action()</code> and predict <code>policy.act(obs, mask)</code> .
3	With probability <code>beta</code> execute the teacher; otherwise execute the learner.
4	Append <code>sample(obs, mask, teacher_label, executed, env.phase, "dagger")</code> .
5	Call <code>env.step(executed)</code> , then repeat from the newly visited input.

Return the row list. Do not call `env.finish()`, append the bot suffix, store the learner action as the target, query the teacher after stepping.

5. `aggregate_dataset(old, new)` [5]

Return a fresh Batch by concatenating observations, targets, and masks along axis 0. Keep all earlier rows, including those from every previous DAgger round. Support discrete targets [N] and continuous targets [N, 2]. Do not mutate the inputs.

$$D_{i+1} = D_i \sqcup \{ (x, \pi_E(x), m) : x \text{ visited in round } i \}$$

Queryable teachers have limitations

Lander uses the Gymnasium heuristic; Chess uses a limited-node Stockfish engine. These are practical label providers, not optimal or omniscient oracles. [3,4]

Human BC labels and bot DAgger labels can differ systematically. A DAgger improvement may reflect better state coverage, different teacher quality, or both. Discuss this confound rather than attributing every gain to the algorithm alone.

Compare data quantity and feedback

Run the default sweep once per variant after completing the five functions. Use the same frozen bank, architecture, and evaluation settings throughout. One training seed is required; three independent training seeds are a useful optional extension. Hyperparameter search is not required.

```
ilr sweep --domain lander-discrete --data data/lander-discrete \
--out runs/lander-discrete
ilr sweep --domain lander-continuous --data data/lander-continuous \
--out runs/lander-continuous
ilr sweep --domain chess --data data/chess --out runs/chess
```

Condition	Data and updates	Evaluate
BC subsets	Lander: 5 / 10 / 50 demonstrations. Chess: 5 / 10 games. Nested human subsets; 12 epochs per model.	Each model on the same 20 held-out seeds
DAGger rounds 1 / 2 / 3	Start BC-50 for Lander or BC-10 for Chess; 5 new prefixes per round; $\beta = .5 / .25 / .125$; aggregate and train 12 epochs	Learner-only prefix after each round
BC extra-training control	Start from the same BC-50 (Lander) or BC-10 (Chess); original data only; approximately match the additional DAGger optimizer updates	Same held-out evaluation protocol

Defaults: Adam learning rate 0.001, minibatch size 128, CPU execution. The control rounds up to complete BC epochs; its update count is recorded and differs by less than one BC epoch. Both methods retain weights but reset the optimizer on each training call. The control helps assess additional optimization, not extra labeling effort.

Holdout discipline and uncertainty

Human collection uses seeds below 10,000; validation starts at 10,000, final testing at 20,000, and DAGger at 30,000. Use validation runs for debugging or any tuning, then freeze settings before final testing. Do not tune to the final test curves. The runner reuses evaluation seeds across conditions and saves raw episode-level outcomes.

Plot means with the supplied 95% episode-bootstrap intervals. Twenty episodes can yield wide or even deceptively narrow intervals for sparse win/loss outcomes. These intervals describe evaluation variation conditional on one trained model; they do not capture training-seed uncertainty. Inspect paired per-seed differences when discussing small improvements.

Outputs to inspect

results.csv contains raw episode metrics; results.json includes summaries and protocols; learning_curves.png plots the main quantity and DAGger comparisons. Each BC directory has a training manifest and checkpoint. Each DAGger round also records cumulative teacher labels, aggregate dataset size, and loss.

Label budget matters

Report new and cumulative oracle labels, not just round count. Fifteen new Lander rollouts can contain many more labeled decisions than fifteen board prefixes. Report collection assistance separately from DAGger queries.

Tell the right story with the metrics

For Lander, define episode return $G_0 = \sum_{t=0}^{T-1} r_t$ with $\gamma = 1$ for this benchmark. Report the empirical mean of G_0 and the fraction with $G_0 \geq 200$. The teacher is never mixed into the evaluation prefix. Action repeat sums both physics-step rewards.

Domain	Primary performance	Required supporting diagnostics
Lander (both)	Mean return; fraction with return ≥ 200	Truncation fraction; continuous control MSE or discrete teacher disagreement
Chess	Hybrid W/D/L and score = win + 0.5 draw	Material at the prefix boundary; move disagreement; capped-game fraction

Material uses pawn=1, knight=bishop=3, rook=5, queen=9, from White's perspective, measured at the prefix boundary. Chess stops at natural game endings or the 160-ply cap; report capped endings separately.

Required figures

Figure 1: three panels showing BC performance: 5/10/50 demonstrations for each Lander variant and 5/10 games for Chess.

Figure 2: three panels showing DAgger performance by round or cumulative oracle labels, with the initial BC baseline (BC-50 for Lander, BC-10 for Chess) and extra-training control.

Figure 3: at least one diagnostic plot from the raw results, such as prefix material or teacher disagreement. Keep return and game-score axes distinct.

Questions your report must answer

Quantity and quality: Which variants benefit from more demonstrations? Could coverage, correlated trajectories, longer episodes, novice improvement during collection, assistance, or label inconsistency explain a trend? Include the actual decision counts.

Actions and representations: How does continuous MSE differ from masked classification? Explain Lander dead zones. Why do Chess openings sparsely cover useful decisions?

Feedback: Does DAgger outperform both its initial BC baseline (BC-50 for Lander, BC-10 for Chess) and the extra-training control? If not, investigate teacher quality, weak learner coverage, mixed data weighting, and finite-sample uncertainty. What distinguishes disagreement from poor task performance?

Discuss failures thoughtfully

Describe one concrete failure or unexpected trend in each domain family. Explain why hybrid final outcomes may hide a weak prefix, and propose one controlled follow-up that distinguishes two plausible explanations. Negative or non-monotonic results can receive full credit when the implementation and analysis are sound.

Finish with a reproducible result

Local setup

Use Python 3.12 and a visible desktop for human collection. Detailed Linux, macOS, and Windows instructions are in README.md. Install Stockfish separately from its official site and set STOCKFISH_EXECUTABLE if needed. Keep its version fixed. A GPU is optional.

```
python3.12 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip setuptools wheel
# Linux CPU; macOS uses the default torch index instead:
python -m pip install torch==2.8.0 \
--index-url https://download.pytorch.org/whl/cpu
python -m pip install -e ".[dev]"
export STOCKFISH_EXECUTABLE=/usr/games/stockfish
ilr doctor
python -m pytest tests/test_infrastructure.py -q
```

If Box2D must compile, install a compiler and SWIG first; the README gives platform-specific commands. The algorithm tests initially fail with NotImplementedError. Implement only the five required functions for the main experiment; the supplied infrastructure tests should pass before you start.

Assessment	Points	Evidence
BC implementation	30	Correct differentiable losses, masking, and minibatch training
Dagger implementation	30	Correct mixture, pre-action teacher labels, and cumulative aggregation
Collection and protocol	8	Three banks (50/50/10), provenance, row counts, data size, frozen nested subsets
Experimental evidence	12	Required curves, control, uncertainty, diagnostics, reproducible settings
Interpretation	16	Thoughtful trends, failures, teacher and metric limitations
Clarity and reproducibility	4	Readable report, citations, commands, manifests, versions

Submit

A 4-6 page report PDF; completed student/algorithms.py; three human NPZ banks (50 discrete Lander, 50 continuous Lander, 10 Chess) and outcome sidecars; result CSV/JSON files; plots; manifests and reproduction commands. Exclude the virtual environment, Stockfish binary, practice/smoke data, caches, and instructor solutions. Keep checkpoints locally; include them only if requested by the course staff.

Before submitting

Verify that each Lander bank contains 50 demonstrations and the Chess bank contains 10 games; assistance is disclosed; smaller subsets are nested; bot suffixes never became training rows; evaluation used the learner alone during its prefix; and every figure identifies the measured quantity and uncertainty. The report template lists the expected structure.

Connect the experiment to the literature

1. Osa et al. (2018)

An Algorithmic Perspective on Imitation Learning. Foundations and Trends in Robotics. [Open source](#)
Read Section 2.1 for design choices: who demonstrates, what is recorded, and how policies are represented.

2. Ross, Gordon, and Bagnell (2011)

A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning. AISTATS. [Open source](#)
Read the motivation and DAgger algorithm. Our short experiment is not a verification of an asymptotic theorem.

3. Farama Foundation

Gymnasium LunarLander-v3 documentation and source. [Open source](#)
Version 1.2.3 supplies the simulator and heuristic. Classroom wrappers add action repeat and the short episode cap.

4. Stockfish developers

Stockfish chess engine, installed separately. [Open source](#)
Record the engine build. A fixed 1,024-node budget supports short runs but does not make the teacher optimal.

Code map

File	Purpose
student/algorithms.py	The five graded functions; exact contracts in docstrings
il_assignment/domains.py	Observation encoding, teachers, action restrictions, and prefix boundaries
il_assignment/ui.py	Interactive controls and collection visualization
il_assignment/data.py	Episode format, provenance, and nested data subsets
il_assignment/model.py	Provided network and observation normalization
il_assignment/experiments.py	Training, DAgger driver, evaluation, summaries, and plots
tests/; report_template.md	Public correctness checks and report scaffold

The instructor release contains separate worked implementations. The student archive excludes them. Third-party code is imported from its published package; no engine binary or external image is bundled. Dependency provenance is documented in THIRD_PARTY.md.